

Desenvolvimento de um Sistema Inteligente de Monitoramento Residencial com Arduino e Processadores ARM

Autor: Wender Gabryell Gomes de Almeida

Instituição: Unic Beira Rio – Universidade de Cuiabá

Curso: Ciência da Computação

Professor: Felipe

1. Introdução

Com o avanço da Internet das Coisas (IoT) e das tecnologias de automação predial, a segurança residencial deixou de ser um sistema passivo para se tornar uma arquitetura ativa de monitoramento em tempo real. O grande desafio atual reside no desenvolvimento de soluções de baixo custo e alta eficiência operacional, capazes de realizar a triagem de dados localmente (computação de borda) e tomar decisões autônomas.

Este artigo apresenta o desenvolvimento e a simulação de um protótipo de Sistema Inteligente de Monitoramento Residencial. O foco principal é a integração de sensores de presença e luminosidade para a tomada de decisões automatizadas em ambientes críticos, utilizando conceitos de programação em linguagem C aplicados a microcontroladores, servindo como base didática para estudos em arquiteturas mais robustas, como a ARM.

2. Fundamentação Teórica

2.1 Microcontroladores e a Arquitetura ARM

Os microcontroladores são sistemas em um único chip (SoC) que integram processador, memória e periféricos de entrada/saída. Enquanto plataformas populares de prototipagem como o Arduino Uno baseiam-se na arquitetura AVR (8-bits), o padrão industrial atual é dominado pela arquitetura **ARM (Advanced RISC Machine)**. Processadores ARM, utilizando o conjunto reduzido de instruções (RISC), oferecem uma eficiência energética e poder de processamento superiores, sendo o núcleo de microcontroladores modernos como a linha STM32 (Cortex-M) e o chip RP2040 da Raspberry Pi. O entendimento do fluxo lógico em C facilita a portabilidade direta de código entre essas arquiteturas através de camadas de abstração de hardware (HAL).

2.2 Programação Embarcada e Linguagem C

A programação embarcada exige o gerenciamento estrito de recursos de hardware. A linguagem C/C++ se consolida neste cenário por permitir o controle direto sobre os registradores do processador, manipulação de interrupções e temporização precisa através de funções nativas de I/O,

garantindo que o ciclo de busca e execução (*fetch-decode-execute*) do processador seja otimizado.

2.3 Sensoriamento e Lógica de Decisão Automatizada

Os sensores atuam como os "órgãos sensoriais" do sistema embarcado. O sensor PIR (*Passive Infrared*) detecta a radiação infravermelha emitida por corpos quentes, identificando movimento. O LDR (*Light Dependent Resistor*) varia sua resistência elétrica conforme a incidência de fótons, permitindo quantificar a luminosidade ambiente. A fusão desses dados permite a implementação de uma lógica condicional booleana, que atua como uma introdução conceitual à Inteligência Artificial (sistemas baseados em regras), simulando a capacidade humana de avaliar um cenário complexo ("Está escuro" **AND** "Há um intruso") antes de disparar uma resposta.

3. Metodologia

A metodologia aplicada consistiu em quatro etapas bem definidas:

1. **Modelagem Lógica:** Criação do algoritmo de controle em blocos condicionais.
2. **Seleção de Hardware:** Escolha de sensores comerciais de alta disponibilidade (PIR HC-SR501 e Módulo LDR).
3. **Simulação Computacional:** Utilização da plataforma **Wokwi** para montagem do circuito digital e validação do firmware em C/C++, eliminando ruídos físicos e problemas de mau contato comuns em matrizes de contato (protoboards).
4. **Análise de Telemetria:** Avaliação do comportamento do processador através do terminal do Monitor Serial em tempo real.

4. Desenvolvimento do Projeto

4.1 Arquitetura e Diagrama do Sistema

O hardware foi projetado conectando os sensores diretamente ao núcleo de processamento. A topologia de conexões está estruturada de acordo com a tabela abaixo:

Componente	Tipo	Pino no Processador	Função Operacional
Sensor PIR	Entrada Digital	Pin 3	Monitoramento de presença em nível lógico Alto (HIGH).

Componente	Tipo	Pino no Processador	Função Operacional
Módulo LDR	Entrada Analógica	Pin A0	Conversão analógico-digital (ADC) da luz de 0 a 1023.
LED Vermelho	Saída Digital	Pin 8	Atuador visual de alerta de intrusão.
Buzzer Piezo	Saída Digital	Pin 9	Atuador sonoro operando em frequência de 1000 Hz.

4.2 Firmware e Fluxo de Execução

O código-fonte foi desenvolvido utilizando estruturas nativas de controle e uma taxa de amostragem definida por interrupção de tempo de 1000 ms.

```
/**
 * SISTEMA INTELIGENTE DE MONITORAMENTO RESIDENCIAL
 * Linguagem: C/C++ Embarcado
 */

const int pirPin = 3;      // Entrada do Sensor PIR
const int ldrPin = A0;     // Entrada Analógica do LDR
const int ledPin = 8;      // Saída do LED de Alerta
const int buzzerPin = 9;   // Saída do Alarme Sonoro

// Limiar adaptativo de luminosidade (Threshold)
const int LIMAR_ESCURO = 400;

void setup() {
    Serial.begin(9600);

    // Configuração dos Modos dos Pinos
    pinMode(pirPin, INPUT);
    pinMode(ledPin, OUTPUT);
    pinMode(buzzerPin, OUTPUT);

    Serial.println("--- Sistema de Monitoramento Residencial Inicializado ---");
}

void loop() {
    // Aquisição de dados periféricos
    int movimento = digitalRead(pirPin);
    int luminosidade = analogRead(ldrPin);

    // Monitoramento Serial (Telemetria)
    Serial.print("Luminosidade Atual: ");
    Serial.print(luminosidade);
    Serial.print(" | Movimento Detectado: ");
    Serial.println(movimento == HIGH ? "SIM" : "NAO");

    // Algoritmo de Tomada de Decisão baseada em Regras (Introdução à IA)
    if (movimento == HIGH && luminosidade < LIMAR_ESCURO) {
        digitalWrite(ledPin, HIGH); // Acionamento do sinalizador visual
        tone(buzzerPin, 1000);      // Emissão de onda quadrada de 1kHz no buzzer
        Serial.println("[ALERTA CRÍTICO] Invasão detectada em ambiente escuro!");
    } else {
        // Retorno ao estado estável de segurança
        digitalWrite(ledPin, LOW);
        noTone(buzzerPin);
    }

    delay(1000); // Ciclo de clock de varredura do sensor
}
```

5. Resultados e Discussões

Os resultados obtidos na plataforma de simulação demonstraram 100% de confiabilidade lógica. O sistema comportou-se de forma estável, dividindo-se em dois estados operacionais perceptíveis na telemetria:

1. **Estado de Vigilância Diurna ou Normalidade:** Com a luminosidade simulada acima de 400 LUX, mesmo que o sensor PIR registrasse nível lógico HIGH (Movimento: 1), o processador bloqueava o disparo dos alarmes. Isso mitiga o problema de falsos positivos durante o dia.
2. **Estado de Disparo Crítico (Noturno):** Quando o controle deslizante do LDR foi configurado para simular o período noturno (valores de ADC abaixo de 400, estabilizando em 250) e o botão *Simulate Motion* do PIR foi acionado, o processador executou as instruções condicionais sem atraso perceptível. O LED alterou seu estado para HIGH e o buzzer gerou a frequência de áudio esperada de 1000 Hz.

A interface serial exibiu com precisão os estados, provando que o fluxo de dados entre os registradores de entrada, a Unidade Lógica e Aritmética (ULA) para a comparação e os registradores de saída ocorreu conforme o esperado para a arquitetura do processador.

6. Conclusão

O projeto atingiu com êxito todos os objetivos propostos. A utilização do simulador Wokwi provou ser uma ferramenta metodológica superior para o ensino de engenharia embarcada, permitindo foco total na arquitetura lógica e na sintaxe da linguagem C, isolando variáveis físicas indesejadas.

A lógica condicional robusta criada funciona perfeitamente como um modelo de tomada de decisão automatizada de baixo custo. Como trabalho futuro, destaca-se que este exato algoritmo pode ser portado diretamente para microcontroladores baseados em processadores **ARM Cortex-M**, utilizando ferramentas como o STM32CubeIDE, expandindo o projeto para uma escala industrial e comercial de automação predial com conectividade Wi-Fi/Bluetooth integrada.

7. Referências

- ARDUINO, CC. *Arduino Language Reference*. Disponível em: <https://www.arduino.cc/reference/en/>.
- MONTEIRO, Mário A. *Organização e Arquitetura de Computadores*. 5. ed. Rio de Janeiro: LTC, 2012.
- STMICROELECTRONICS. *Introduction to ARM Cortex-M Microcontrollers*. Disponível em: <https://www.st.com>.
- WOKWI. *Wokwi Arduino Simulator User Guide*. Disponível em: <https://docs.wokwi.com>.